P2592R1
Hashing support for std::chrono value classes

Published Proposal, 2022-06-30

Author:
Giuseppe D'Angelo

Audience:
LEWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

We propose to add `std::hash` specializations for the value classes in the `std::chrono` namespace.

Table of Contents

§ 1. Changelog

- R1
  - Incorporated the feedback from a LEWG teleconference.
  - Added a note to the specializations for the civil calendar classes, highlighting the fact that it's possible to build objects of those classes with unspecified values. If so, hashing them also yields unspecified values.
  - Added notes about support for heterogeneous hashing (or lack thereof).
- R0
  - First submission

§ 2. Motivation and Scope

The time library defines several value classes and class templates in the `std::chrono` namespace. All of the following are copyable and comparable for equality (and some are also default constructible, therefore satisfying `regular`):

- `duration`
- `time_point`
- `day`
- `month`
- `year`
- `weekday`
- `weekday_indexed`
- `weekday_last`

- `month_day`
- `month_day_last`
- `month_weekday`
- `month_weekday_last`
- `year_month`
- `year_month_day`
- `year_month_day_last`
- `year_month_weekday`
- `year_month_weekday_last`
- `zoned_time`
- `leap_second`

Unfortunately, **none** of these classes comes with an enabled `std::hash` specialization. This means that for instance it's not possible to store them in the Standard Library unordered associative containers:

```
std::unordered_set<std::chrono::milliseconds> unique_measurements; // ERROR
```

Of course users can work around this limitation by providing a custom hash, for instance something like this:

```
struct duration_hash
{
    template <class Rep, class Period>
    constexpr auto operator()(const std::chrono::duration<Rep, Period> &d)
        noexcept(/* ... */)
    {
        std::hash<Rep> h;
        return h(d.count());
    }
};

std::unordered_set<std::chrono::milliseconds, duration_hash> unique_measure
```

This limitation seems however to be unnecessarily vexing. Most of the types listed above can actually have a straightforward implementation for their hashing function, and there's little justification for the Standard Library itself not to provide it.

(On the contrary: we argue that it's the Standard Library responsability to provide hashing for all of its movable types that can be compared for equality.)

The only case where a slightly more challenging implementation may be required is `zoned_time`, because it may need to mix the hash for the time zone pointer with the hash for the time point. This is not however different from "prior art"; for instance, the implementation of `hash<error_code>` has to solve the same problem.

§ 3. Impact On The Standard

This proposal is a pure library addition.

This proposal does not depend on any other library extensions.

This proposal does not require any changes in the core language.

§ 4. Design Decisions

§ 4.1. What about `time_zone`?

While `time_zone` is comparable for equality, it is not a regular type because it is not copyable. Users that need to store `time_zone` objects typically use *pointers* to them. As such, we do not see the need to add a `hash<time_zone>` specialization, and we are not proposing it here.

The same reasoning applies to `time_zone_link`.

§ 4.2. Support for heterogeneous hashing

The `hash` specializations that we are proposing do not directly support heterogeneous hashing, as they lack the `hash<T>::is_transparent` type ([unord.req.general/10.7]) which is needed to enable the heterogeneous overloads of `find`, `erase`, and so on. This is consistent with all the other `hash` specializations available in the Standard Library, which are not enabled for heterogeneous hashing.

This is an important safety net, as hashing objects of different datatypes may yield results that would violate the hashing requirements. For instance, given

```
std::chrono::milliseconds a = 1000ms;
std::chrono::seconds      b = 1s;
```

then `a` and `b` do compare equal, but it is not guaranteed that hashing the two objects would produce the same value (on any "reasonable" implementation they would actually produce different values).

4.2.1. Implicit conversions

The lack of direct support for heterogeneous hashing does not mean that one cannot simply build on top of the existing implicit conversions for `std::chrono` classes. Consider this example:

```
std::unordered_set<std::chrono::milliseconds> unique_measurements;
unique_measurements.insert(1000ms);
assert(unique_measurements.contains(1s)); // OK
```

With the proposed wording, the example compiles and works as expected; the `assert` does not fire. What is happening here is that the `std::chrono::second` object passed to `contains` is being implictly converted to a `std::chrono::milliseconds` object, which is then used by `contains`.

Having implicit conversions between different specializations is is a major design choice of `chrono` class templates, specifically for `duration`, `time_point` and `zoned_time` (the last two are indeed built on top of `duration`); the converting constructors are constrained to work only under precise conditions. The same constraints would apply in code like the one exemplified above.

4.3. Handling the unspecified state of the calendaring classes

The calendaring classes introduced in C++20 can have an unspecified state if built with out-of-range values. For instance:

```
std::chrono::day d(0u); // unspecified; not undefined behavior
```

What should happen if one tries to hash `d`? We claim that the operation should simply give an unspecified result, and *not* undefined behavior.

The `chrono` facilities deliberately avoid triggering undefined behavior in many places, preferring to return unspecified values instead. Let's extend the example:

```
std::chrono::day d(0u);                         // unspecified
std::chrono::month_day md = April / d;          // unspecified because of 'd'
std::chrono::year_month_day ymd = 2022 / md;    // unspecified because of 'md

// conversion, [time.cal.ymd.members]
std::chrono::sys_days sd = ymd;                 // unspecified
```

We believe that the proposed `hash` specializations should follow this precedent and also return unspecified values.

However it's important to remark that this does not mean that an implementation is allowed to return "anything": hashing objects that compare equal must still yield the same value. In other words, the hashing *requirements* must still be fulfilled, even in the presence of unspecified inputs.

Therefore:

```
std::chrono::day d(0u); // Unspecified...

assert(d == d);         // but whatever the value contained inside 'd' is,

auto h1 = std::hash<std::chrono::day>()(d);
auto h2 = std::hash<std::chrono::day>()(d);

assert(h1 == h2);       // and therefore this must hold as well.
```

or generalizing:

```
std::chrono::day d1(0u);
std::chrono::day d2(1234u);

auto h1 = std::hash<std::chrono::day>()(d1);
auto h2 = std::hash<std::chrono::day>()(d2);

// d1, d2, h1, h2 are all unspecified.
// 'd1' is not guaranteed to be equal to (or different from) 'd2';
```

```
// but if it is equal, then the hashes are equal:
if (d1 == d2)
  assert(h1 == h2);
```

§ 5. Technical Specifications

All the proposed changes are relative to [N4910].

§ 5.1. Feature testing macro

In [version.syn], modify

```
#define __cpp_lib_chrono 201907L YYYYMML // also in <chrono>
```

by replacing the existing value with the year and month of adoption of the present proposal.

§ 5.2. Proposed wording

Add the following at the end of [time.syn]:

```
namespace std {
  // ???, hash support
  template<class T> struct hash;
  template<class Rep, class Period> struct hash<chrono::duration<Rep, Per
  template<class Clock, class Duration> struct hash<chrono::time_point<Cl
  template<> struct hash<chrono::day>;
  template<> struct hash<chrono::month>;
  template<> struct hash<chrono::year>;
  template<> struct hash<chrono::weekday>;
  template<> struct hash<chrono::weekday_indexed>;
  template<> struct hash<chrono::weekday_last>;
  template<> struct hash<chrono::month_day>;
  template<> struct hash<chrono::month_day_last>;
  template<> struct hash<chrono::month_weekday>;
  template<> struct hash<chrono::month_weekday_last>;
  template<> struct hash<chrono::year_month>;
  template<> struct hash<chrono::year_month_day>;
  template<> struct hash<chrono::year_month_day_last>;
  template<> struct hash<chrono::year_month_weekday>;
  template<> struct hash<chrono::year_month_weekday_last>;
  template<class Duration, class TimeZonePtr> struct hash<chrono::zoned_t
  template<> struct hash<chrono::leap_second>;
}
```

Add a new subclause after [time.parse], with the following content:

??? Hash support [time.hash]

```
template<class Rep, class Period> struct hash<chrono::duration<Rep, Perio
```

(1) Letting D be `chrono::duration<Rep, Period>`, the specialization `hash<D>` is enabled ([unord.hash]) if and only if `hash<Rep>` is enabled. When enabled, for an object `d` of type `D`, `hash<D>()(d)` evaluates to the same value as `hash<Rep>()(d.count())`. The member functions are not guaranteed to be `noexcept`.

```
template<class Clock, class Duration> struct hash<chrono::time_point<Cloc
```

(2) Letting TP be `chrono::time_point<Clock, Duration>`, the specialization `hash<TP>` is enabled ([unord.hash]) if and only if `hash<Duration>` is enabled. When enabled, for an object `tp` of type `TP`, `hash<TP>()(tp)` evaluates to the same value as `hash<Duration>()(tp.time_since_epoch())`. The member functions are not guaranteed to be `noexcept`.

```
template<> struct hash<chrono::day>;
template<> struct hash<chrono::month>;
template<> struct hash<chrono::year>;
template<> struct hash<chrono::weekday>;
template<> struct hash<chrono::weekday_indexed>;
template<> struct hash<chrono::weekday_last>;
template<> struct hash<chrono::month_day>;
template<> struct hash<chrono::month_day_last>;
template<> struct hash<chrono::month_weekday>;
template<> struct hash<chrono::month_weekday_last>;
template<> struct hash<chrono::year_month>;
template<> struct hash<chrono::year_month_day>;
template<> struct hash<chrono::year_month_day_last>;
template<> struct hash<chrono::year_month_weekday>;
template<> struct hash<chrono::year_month_weekday_last>;
```

(3) The specialization is enabled ([unord.hash]).

(4) Given an object `h` of type `hash<Key>` (where `hash<Key>` is one of the specializations listed above) and an object `k` of type (possibly cv-qualified) `Key`, if `k.ok() == false`, then the evaluation of `h(k)` returns an unspecified value.

[*Note* 1: These `hash<Key>` specializations still meet the *Cpp17Hash* requirements, even for objects `k` such that `k.ok() == false`. — *end note*]

```
template<class Duration, class TimeZonePtr> struct hash<chrono::zoned_tim
```

(5) Letting ZT be `chrono::zoned_time<Duration, TimeZonePtr>`, the specialization `hash<ZT>` is enabled ([unord.hash]) if and only if `hash<Duration>` is enabled and `hash<TimeZonePtr>` is enabled. The member functions are not guaranteed to be `noexcept`.

```
template<> struct hash<chrono::leap_second>;
```

(6) The specialization is enabled ([unord.hash]).

6. Acknowledgements

Thanks to KDAB for supporting this work.

Thanks to Howard Hinnant for designing `<chrono>` and for discussing why hash support had not been added in the past.

All remaining errors are ours and ours only.

References

Informative References

[N4910]
Thomas Köppe. *Working Draft, Standard for Programming Language C++*. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/n4910.pdf